Kubernetes — Workloads, Services, ConfigMaps & Secrets

These notes cover the practical Kubernetes setup, Pods, workload controllers, Services, ConfigMaps, Secrets, and how these components come together to form a basic Kubernetes application architecture.

---

1. Kubernetes Practical Setup

For local learning, we can run Kubernetes using **Minikube** inside an Ubuntu VM.

The basic stack is:

```
Ubuntu VM
   ↓
Docker
   ↓
Minikube
   ↓
Kubernetes Cluster
```

Install `kubectl`

`kubectl` is the command-line tool used to communicate with a Kubernetes cluster.

```
sudo snap install kubectl --classic
```

Verify:

```
kubectl version --client
```

Check Ubuntu Architecture

```
dpkg --print-architecture
```

On an ARM-based machine such as an Apple Silicon Mac running Ubuntu through Multipass:

```
arm64
```

Install Minikube

Download the ARM64 package:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_arm64.de
b
```

Install it:

```
sudo dpkg -i minikube_latest_arm64.deb
```

Remove the installer:

```
rm minikube_latest_arm64.deb
```

Verify:

```
minikube version
```

Verify Docker

```
docker --version
```

```
docker ps
```

Docker will be used as the driver on which Minikube creates the Kubernetes node.

Configure Minikube to Use Docker

```
minikube config set driver docker
```

Conceptually:

```
Minikube
   ↓
Docker Driver
   ↓
Kubernetes Node
```

Once configured, Minikube remembers Docker as the default driver.

Start Minikube

For an Ubuntu VM with roughly 4 GB RAM:

```
minikube start --memory=2500mb --preload=false
```

Check its status:

```
minikube status
```

Expected components include:

```
host: Running
kubelet: Running
```

```
apiserver: Running
kubeconfig: Configured
```

---

2. Exploring the Kubernetes Cluster

Check Cluster Information

```
kubectl cluster-info
```

This confirms that:

```
kubectl
   ↓
Kubernetes API Server
   ↓
Cluster
```

`kubectl` communicates with Kubernetes primarily through the **API Server**.

View Nodes

```
kubectl get nodes
```

Example:

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   ...   ...
```

With Minikube, there may be only one node.

For learning purposes, the same node can host:

```
Control Plane Components
+
Application Workloads
```

In larger production environments, control-plane and worker responsibilities are normally distributed differently.

For more details:

```
kubectl get nodes -o wide
```

This shows additional information such as:

```
INTERNAL-IP
OS-IMAGE
KERNEL-VERSION
CONTAINER-RUNTIME
```

`get` vs `o wide`

```
kubectl get ...
```

provides a summary.

```
kubectl get ... -o wide
```

provides additional details.

---

3. View Kubernetes System Components

```
kubectl get pods -n kube-system
```

Important Kubernetes components visible here include:

```
API Server
etcd
Scheduler
Controller Manager
kube-proxy
CoreDNS
```

This connects the theoretical Kubernetes architecture with the actual running cluster.

---

4. Creating Our First Pod

A **Pod** is the smallest deployable unit in Kubernetes.

Instead of directly asking Kubernetes to run a container, we normally run the container inside a Pod.

```
Pod
 ↓
Container
```

Create an Nginx Pod:

```
kubectl run demo-pod \
  --image=nginx:alpine \
  --port=80
```

Output:

```
pod/demo-pod created
```

Check the Pod:

```
kubectl get pods
```

Initially, we may see:

```
NAME       READY   STATUS              RESTARTS
demo-pod   0/1     ContainerCreating   0
```

After Kubernetes finishes creating the container:

```
NAME       READY   STATUS    RESTARTS
demo-pod   1/1     Running   0
```

Understanding `READY 1/1`

```
READY

1 / 1
↑   ↑
|   └── Total containers
└────── Ready containers
```

So:

```
demo-pod
   ↓
1 Nginx container
```

and that container is ready.

Check Where the Pod Is Running

```
kubectl get pods -o wide
```

Example:

```
NAME       READY   STATUS    IP          NODE
demo-pod   1/1     Running   10.x.x.x    minikube
```

Architecture:

```
Kubernetes Cluster
        ↓
Node: minikube
        ↓
Pod: demo-pod
        ↓
Container: nginx
```

The creation flow can be thought of as:

```
kubectl run
     ↓
API Server
     ↓
Scheduler
     ↓
Selects a Node
     ↓
kubelet
     ↓
Container Runtime
     ↓
Nginx Container
```

Delete the Pod

```
kubectl delete pod demo-pod
```

Output:

```
pod "demo-pod" deleted
```

Verify:

```
kubectl get pods
```

The Pod should no longer exist.

---

5. Why Running Individual Pods Is Not Enough

Suppose we create:

```
Pod
 └── Spring Boot Container
```

The application is running.

But the Pod may disappear because of:

```
Node problem
Pod deletion
Pod eviction
Application redeployment
Infrastructure changes
```

If nothing is managing that Pod, Kubernetes has no desired state telling it to recreate another one.

When we run:

```
kubectl run my-app ...
```

we are effectively asking Kubernetes:

> Create this Pod.

But in production, that is usually not what we actually care about.

Instead, our requirement may be:

> Always keep three instances of my application running.

That is a completely different requirement.

---

6. Desired State vs Actual State

Suppose we need three application instances.

```
Desired replicas = 3
Actual replicas  = 2
```

Kubernetes should automatically notice:

```
Desired State
      ↓
   Compare
      ↓
Actual State
```

and create another Pod.

This is one of the most important ideas in Kubernetes.

Controllers continuously try to make:

```
Actual State
```

match:

```
Desired State
```

---

7. Workload Controllers

A **workload controller** manages Pods according to some desired behavior.

Instead of simply saying:

```
Run this Pod
```

we can express requirements such as:

```
Always maintain 3 interchangeable Pods.
```

or:

```
Run one Pod on every Node.
```

or:

```
Run 3 database Pods and preserve their identities.
```

Different requirements are handled by different controllers.

The four important controllers are:

```
ReplicaSet
Deployment
DaemonSet
StatefulSet
```

---

8. Labels and Selectors

Before understanding workload controllers, we need to understand **labels**.

Suppose Kubernetes has:

```
Pod xyz128
Pod abc921
```

```
Pod pqr773
```

A controller needs some way to identify which Pods belong to a particular application.

Kubernetes uses **labels**.

Example:

```
labels:
  app: shop-api
```

Multiple Pods can have the same label:

```
Pod A → app=shop-api
Pod B → app=shop-api
Pod C → app=shop-api
```

A controller can then use a selector:

```
Find Pods where:

app = shop-api
```

Label

A tag or identity attached to a Kubernetes object.

Selector

A query used to find Kubernetes objects with matching labels.

```
Labels
   ↓
Identify objects

Selectors
   ↓
Find matching objects
```

Both **controllers** and **Services** make heavy use of labels and selectors.

---

9. ReplicaSet

A ReplicaSet solves a simple problem:

> Always maintain a specified number of matching Pod replicas.

Suppose:

```
Desired replicas = 3
```

Initially:

```
Actual replicas = 0
```

ReplicaSet creates three Pods:

```
ReplicaSet
   ↓
Pod A
Pod B
Pod C
```

Now:

```
Desired = 3
Actual  = 3
```

Everything matches.

What If a Pod Dies?

Suppose:

```
Pod A ✅
Pod B ❌
Pod C ✅
```

Now:

```
Desired = 3
Actual  = 2
```

ReplicaSet notices the difference and creates another Pod.

```
Pod D
```

Now:

```
Pod A
Pod C
Pod D

Actual = 3
```

ReplicaSet Does Not Resurrect the Exact Same Pod

Suppose:

```
shop-api-a8j2
```

dies.

ReplicaSet may create:

```
shop-api-k9f4
```

instead.

For stateless application Pods, Kubernetes generally treats Pods as replaceable.

What matters is:

```
app=shop-api
```

not the identity of one particular Pod.

---

10. ReplicaSet YAML Mental Model

```
apiVersion: apps/v1
kind: ReplicaSet

metadata:
  name: shop-api-rs

spec:
  replicas: 3

  selector:
    matchLabels:
      app: shop-api

  template:
    metadata:
      labels:
        app: shop-api

    spec:
      containers:
        - name: shop-api
          image: coderarmy/shop-api:v1
```

Read this as English.

`kind`

```
kind: ReplicaSet
```

means:

> Create a ReplicaSet.

`replicas`

```
replicas: 3
```

means:

> Maintain three Pods.

`selector`

```
app: shop-api
```

means:

> These are the Pods managed by this ReplicaSet.

`template`

The template tells Kubernetes how to create new Pods.

```
image: coderarmy/shop-api:v1
```

Conceptually:

```
If another Pod is needed,
build it using this template.
```

This is the **Pod template**.

---

11. Why ReplicaSet Alone Is Not Enough

Suppose ReplicaSet currently maintains:

```
shop-api:v1
shop-api:v1
shop-api:v1
```

Now version 2 is released:

```
shop-api:v2
```

We probably want:

```
v1 v1 v1
    ↓

v2 v1 v1
    ↓

v2 v2 v1
    ↓

v2 v2 v2
```

Now the requirement is no longer just:

> Keep three Pods alive.

We also need:

```
Version management
Rolling updates
Rollback
Deployment history
Controlled replacement
```

This is where **Deployment** becomes important.

---

12. Deployment

For normal stateless applications, **Deployment** is usually the workload controller we use.

Common examples:

```
Spring Boot REST API
Node.js backend
React frontend
Python FastAPI service
Authentication service
Payment API
Notification service
```

---

13. Deployment vs ReplicaSet

Think of the hierarchy:

```
Deployment
     ↓
ReplicaSet
     ↓
Pods
     ↓
Containers
```

A Deployment does not primarily manage individual Pods directly.

Conceptually:

```
Deployment:
"I want version 2 with 3 replicas."

              ↓

ReplicaSet:
"I will maintain those 3 Pods."

              ↓

             Pods
```

ReplicaSet's main concern

```
How many Pods should exist?
```

Deployment's higher-level concerns

```
Which application version should run?

How should updates happen?

Should we rollback?

How should ReplicaSets be managed over time?
```

---

14. Why We Usually Use Deployment Instead of ReplicaSet Directly

ReplicaSet gives us replica management.

Deployment gives us replica management **plus application lifecycle management**.

Instead of:

```
You
 ↓
ReplicaSet
 ↓
Pods
```

we normally use:

```
You
 ↓
Deployment
 ↓
ReplicaSet
```

```
 ↓
Pods
```

A useful rule to remember:

> **Understand ReplicaSet. Use Deployment for ordinary applications.**

---

15. Deployment Example

Suppose:

```
Deployment: shop-api

replicas = 3
image    = shop-api:v1
```

Kubernetes creates:

```
Deployment
    ↓
ReplicaSet-v1
    ↓
Pod v1
Pod v1
Pod v1
```

---

16. Rolling Updates

Now release version 2.

Change:

```
shop-api:v1
```

to:

```
shop-api:v2
```

The Deployment notices that the Pod template changed.

It creates a new ReplicaSet.

```
Deployment
    │
    ├── Old ReplicaSet
    │       ↓
    │   v1 v1 v1
    │
    └── New ReplicaSet
            ↓
            v2
```

The transition happens gradually:

```
Old RS: 3
New RS: 0
```

then:

```
Old RS: 2
New RS: 1
```

then:

```
Old RS: 1
New RS: 2
```

finally:

```
Old RS: 0
New RS: 3
```

This is called a **Rolling Update**.

17. Deployment Rollback

Suppose:

```
v1 ✅
```

Then version 2 is deployed:

```
v2 ❌
```

If version 2 contains a serious problem, Deployment allows us to return to a previous deployment revision.

Conceptually:

```
v2
 ↓
Rollback
 ↓
v1
```

This is one of the major reasons Deployment exists above ReplicaSet.

18. Scaling a Deployment

Suppose:

```
replicas = 3
```

Traffic increases.

Change:

```
replicas = 10
```

Now:

```
Deployment
    ↓
ReplicaSet
    ↓
10 Pods
```

Scaling is different from a version rollout.

Changing:

```
replicas: 3 → 10
```

does not mean the application version changed.

Only the desired number of instances changed.

---

19. When Should We Use Deployment?

Ask:

> Are these application Pods interchangeable?

Suppose:

```
shop-api Pod A
shop-api Pod B
shop-api Pod C
```

If any incoming request can go to any one of them:

```
User
 ↓
```

```
A / B / C
```

and it does not matter which specific Pod serves the request, a Deployment is usually appropriate.

These are generally **stateless workloads**.

Stateless does not mean the entire system has no data.

A Spring Boot Pod may still communicate with:

```
MySQL
Redis
MongoDB
S3
```

The important point is:

> The individual application Pod does not require a persistent unique identity.

---

20. DaemonSet

Now consider a different requirement.

Suppose the cluster has:

```
Node A
Node B
Node C
```

We need a log collection agent running on every node.

```
Node A → Log Agent
Node B → Log Agent
Node C → Log Agent
```

Could we create a Deployment with:

```
replicas = 3
```

?

Not necessarily.

The scheduler could theoretically place:

```
Node A → Pod 1
Node A → Pod 2
Node B → Pod 3
Node C → none
```

We still have three Pods, but our requirement is not satisfied.

Our requirement is:

> Run one relevant Pod on every Node.

That is what **DaemonSet** is designed for.

---

21. How DaemonSet Works

A DaemonSet ensures that all, or a selected subset of, Kubernetes Nodes run a copy of a Pod.

Example:

```
Node A
 └── log-agent

Node B
 └── log-agent

Node C
 └── log-agent
```

Now suppose:

```
Node D
```

is added.

The DaemonSet automatically creates:

```
Node D
 └── log-agent
```

When qualifying Nodes appear, corresponding Pods can be added automatically.

---

22. Common DaemonSet Use Cases

DaemonSets are generally useful for **node-level infrastructure**.

Examples:

```
Log collection agent

Monitoring agent

Storage agent

Networking component

Security / observability agent

Hardware driver / helper
```

---

23. StatefulSet

Now consider workloads such as:

```
MySQL
PostgreSQL
Kafka
```

```
Cassandra
ZooKeeper
```

With a normal application Deployment, Pods may be interchangeable.

If:

```
Pod A
```

disappears, creating:

```
Pod D
```

may be perfectly fine.

But database replicas may need:

```
mysql-0
mysql-1
mysql-2
```

where each instance has its own identity and storage.

Now the Pods are no longer completely interchangeable.

---

24. Why Deployment Is Awkward for Stateful Workloads

Imagine three database Pods:

```
Database Pod abc128
Database Pod xyz629
Database Pod pqx420
```

If one disappears, Deployment may create another randomly identified Pod.

But a stateful workload may require:

```
DB replica 0
     ↓
Disk 0
```

and:

```
DB replica 1
     ↓
Disk 1
```

We may therefore need:

```
Stable identity
Stable storage relationship
Predictable names
Ordered startup / shutdown
```

That is the purpose of **StatefulSet**.

---

25. StatefulSet Identity

Suppose:

```
StatefulSet: database
replicas: 3
```

Pods receive predictable identities:

```
database-0
database-1
database-2
```

Compare this with Deployment-style Pods:

```
web-k72sm
web-x82qp
web-j91aa
```

For the web Pods, we usually do not care which one is which.

For StatefulSet:

```
mysql-0
mysql-1
mysql-2
```

the individual identity matters.

If:

```
mysql-1
```

fails, Kubernetes recreates the workload while maintaining:

```
mysql-1
```

as the StatefulSet identity.

---

26. StatefulSet and Stable Storage

Conceptually:

```
mysql-0 → Disk 0
mysql-1 → Disk 1
mysql-2 → Disk 2
```

If:

```
mysql-1
```

disappears, a replacement `mysql-1` can reconnect with the storage associated with that identity.

```
new mysql-1
     ↓
Disk 1
```

This is one of the main reasons StatefulSet exists.

---

27. Ordered Stateful Workloads

Some stateful applications care about startup and shutdown order.

For example:

```
database-0
    ↓
must become ready

before

database-1
```

and:

```
database-1
    ↓
before

database-2
```

StatefulSets can support ordered deployment, scaling, and updates.

---

28. Workload Controller Summary

Controller	Fundamental Question
ReplicaSet	How do I maintain N identical Pods?
Deployment	How do I deploy, update and manage stateless application replicas?
DaemonSet	How do I run a Pod on every relevant Node?
StatefulSet	How do I manage Pods whose identity, storage or order matters?

Typical examples:

```
Spring Boot REST API
        ↓
Deployment
```

```
Node Log Collector
        ↓
DaemonSet
```

```
Database Cluster
        ↓
StatefulSet
```

For ordinary stateless applications:

```
Deployment
   ↓
ReplicaSet
   ↓
Pods
```

---

29. Kubernetes Services

Workload controllers solve:

> How do we keep application Pods running?

But they introduce another problem.

Suppose Deployment maintains three Pods:

```
shop-api
replicas = 3
```

Each Pod has its own IP:

```
Pod A → 10.244.0.11
Pod B → 10.244.0.12
Pod C → 10.244.0.13
```

A frontend needs to call the backend.

Which IP should it use?

---

30. Why Clients Should Not Depend on Pod IPs

Suppose the frontend calls:

```
10.244.0.11
```

Pod A crashes.

ReplicaSet replaces it with:

```
Pod D → 10.244.0.27
```

The old IP:

```
10.244.0.11
```

is no longer useful.

Pods are **ephemeral**.

Their network identities are not a good permanent contract for clients.

31. The Actual Networking Requirement

The frontend does not really want:

> Talk to Pod A.

It wants:

> Talk to the `shop-api` application.

We need a stable abstraction in front of the changing Pods.

That abstraction is a:

Service

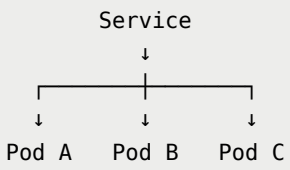
32. What Is a Kubernetes Service?

A Service provides a stable network abstraction in front of a dynamic set of Pods.

Instead of:

```
Frontend
   ↓
Pod A IP
```

we use:



Pods may:

```
die
restart
change IP
scale up
scale down
```

while the Service remains the stable access point.

---

33. How Does a Service Find Pods?

Services use **labels and selectors**.

Pods:

```
Pod A → app=shop-api
Pod B → app=shop-api
Pod C → app=shop-api
```

Service selector:

```
app=shop-api
```

Result:

```
Service
selector: app=shop-api
       ↓
Pod A ✅
Pod B ✅
Pod C ✅
```

A Pod with:

```
app=payment-api
```

does not match and is ignored.

This is why labels and selectors are fundamental in Kubernetes.

---

34. What Happens When Pods Change?

Initially:

```
Service
   ↓
A B C
```

Suppose Pod B disappears and Pod D replaces it.

The eligible endpoints become:

```
Service
   ↓
A C D
```

The client continues communicating with the Service.

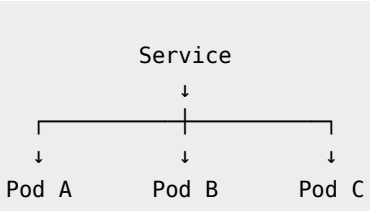
It does not need to track the individual Pod changes.

---

35. Service and Traffic Distribution

Suppose many requests arrive.

The Service can route connections across eligible backend Pods.



A Service should primarily be understood as:

```
Stable service discovery
+
```

```
Routing toward eligible Pods
```

rather than as a sophisticated application-aware load balancer.

---

36. Service DNS

Services also provide a stable DNS-based way for workloads to find each other.

Suppose we create a Service named:

```
shop-api
```

Other workloads in the appropriate namespace/context can communicate using something like:

```
http://shop-api
```

Flow:

```
Frontend Pod
     ↓
shop-api
     ↓
Kubernetes DNS
     ↓
Service
     ↓
Backend Pods
```

This is extremely useful in microservice architectures.

Instead of configuring:

```
PAYMENT_SERVICE=10.244.0.13
```

we can use a stable service identity such as:

```
PAYMENT_SERVICE=http://payment-service
```

---

37. Kubernetes Service Types

The three important Service types for this level are:

```
ClusterIP
NodePort
LoadBalancer
```

---

38. ClusterIP

`ClusterIP` is the default Service type.

It is intended for communication from inside the Kubernetes cluster.

Example:

```
Frontend Deployment
        ↓
Backend Service
        ↓
Backend Pods
```

Architecture:

```
        KUBERNETES CLUSTER

Frontend Pod
     ↓
Backend Service
   ClusterIP
     ↓
┌────┼────┐
↓    ↓    ↓
Pod  Pod  Pod
```

This is commonly used for internal service-to-service communication.

ClusterIP Example

Consider an e-commerce system:

```
Frontend
Order Service
Payment Service
Inventory Service
MySQL
```

Services such as:

```
payment-service
inventory-service
order-service
```

could use ClusterIP.

Example:

```
Order Service
     ↓
payment-service
     ↓
Payment Pods
```

There is no need to expose every internal microservice directly to the Internet.

---

39. NodePort

Sometimes we want traffic from outside the cluster.

A basic option is **NodePort**.

Kubernetes exposes the Service through a port on the Node.

```
Outside Client
      ↓
NodeIP:NodePort
      ↓
Service
      ↓
Pods
```

Example:

```
Node IP  = 192.168.1.20
NodePort = 30080
```

The application could then be reached through:

```
192.168.1.20:30080
```

Traffic is forwarded through the Service to matching Pods.

---

40. LoadBalancer

In a cloud environment, we may want:

```
Internet
    ↓
External Load Balancer
    ↓
Kubernetes Service
    ↓
Pods
```

A Service of type:

```
type: LoadBalancer
```

asks the environment or cloud integration to provision or connect an external load balancer where supported.

Conceptually:

```
AWS / GCP / Azure Load Balancer
              ↓
       Kubernetes Service
              ↓
        Pod   Pod   Pod
```

---

41. Service Type Summary

```
ClusterIP
   ↓
Internal cluster access
```

```
NodePort
   ↓
Expose through a Node port
```

```
LoadBalancer
   ↓
External load balancer in front
```

---

42. `port`, `targetPort` and `nodePort`

These three ports can initially be confusing.

Suppose the application inside the container listens on:

```
8080
```

`targetPort`

`targetPort` is the port where the application inside the Pod is listening.

```
targetPort: 8080
```

`port`

`port` is the port exposed by the Service.

Example:

```
port: 80
```

Flow:

```
Client
  ↓
Service:80
  ↓
Pod:8080
```

`nodePort`

`nodePort` is used when exposing through NodePort.

Example:

```
nodePort: 30080
```

Flow:

```
External Client
       ↓
NodeIP:30080
       ↓
Service:80
```

```
       ↓
Pod:8080
```

So the mental model is:

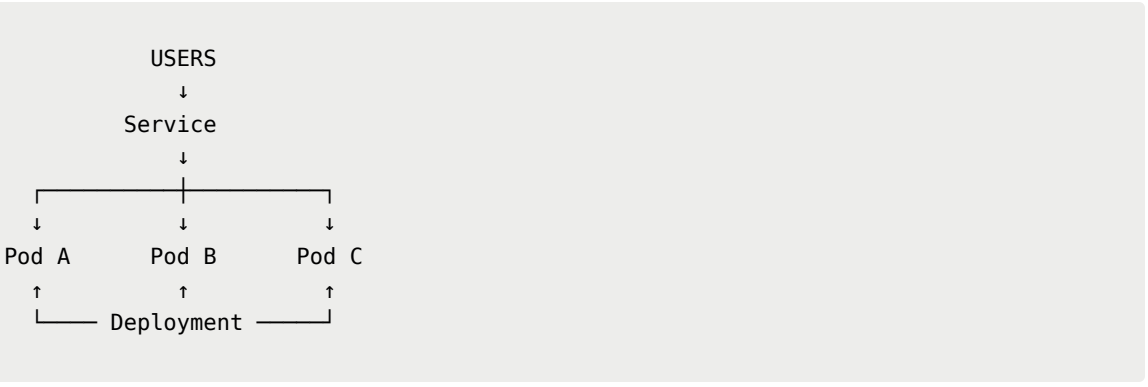
```
nodePort
   ↓
port
   ↓
targetPort
```

Example:

```
30080
  ↓
 80
  ↓
8080
```

---

43. Complete Service + Deployment Mental Model



Remember that these are two separate relationships.

Deployment

```
Deployment
   ↓
manages Pods
```

Service

```
Service
   ↓
discovers matching Pods using labels
```

---

44. What Happens When We Scale?

Suppose Deployment changes from:

```
3 Pods
```

to:

```
10 Pods
```

Each Pod receives:

```
app=shop-api
```

The Service selector remains:

```
app=shop-api
```

Therefore:

```
Service
   ↓
```

```
10 eligible Pods
```

The client does not need to change anything.

This combination is one of the most important ideas in Kubernetes:

```
Deployment
   ↓
Dynamic application instances

Service
   ↓
Stable network identity
```

---

45. ConfigMaps and Secrets

Our application can now:

```
run
scale
update
be reached reliably
```

But we still need to solve application configuration.

Suppose a Spring Boot application requires:

```
server.port=8080
spring.datasource.url=jdbc:mysql://mysql:3306/shop
feature.payment.enabled=true
logging.level.root=INFO
```

It may also require:

```
DB_USERNAME=admin
DB_PASSWORD=abc123
```

Where should this information live?

46. Why Configuration Should Not Be Baked into the Image

One option would be:

```
shop-api:v1
```

containing:

```
Application Code
+
Database Address
+
Environment Settings
+
Passwords
```

This creates problems.

Consider:

```
Development
Testing
Staging
Production
```

The application code may be identical, while configuration differs.

Example:

```
Development DB:
localhost
```

```
Testing DB:
test-db
```

```
Production DB:
prod-database.internal
```

If configuration is embedded in the image, we may end up creating:

```
shop-api-dev:v1
shop-api-test:v1
shop-api-prod:v1
```

Now different environments are not running the exact same artifact.

A better model is:

```
One Image
shop-api:v1
      ↓
Different Configuration
      ↓
Development / Testing / Production
```

---

47. Separate Code from Configuration

Conceptually:

```
Docker Image
=
Application + Runtime
```

while:

```
Configuration
=
Environment-dependent information
```

Instead of:

```
Image
+
DB_HOST=production-db
```

we want:

```
Image
+
External Configuration
```

Kubernetes provides:

```
ConfigMap
Secret
```

for this purpose.

---

48. ConfigMap

A **ConfigMap** stores non-confidential configuration.

Examples:

```
APP_MODE=production
LOG_LEVEL=INFO
PAYMENT_ENABLED=true
DATABASE_HOST=mysql-service
```

These values are not passwords or private credentials.

Conceptually:

```
Docker Image
     +
ConfigMap
```

A Pod can then consume values from the ConfigMap.

```
Pod
 ├── Container
 │
 └── ConfigMap
```

This allows the same image to run with different configuration in different environments.

---

49. ConfigMap Example

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: shop-config

data:
  APP_MODE: "production"
  LOG_LEVEL: "INFO"
  DATABASE_HOST: "mysql-service"
```

Conceptually:

```
ConfigMap: shop-config

APP_MODE=production
LOG_LEVEL=INFO
DATABASE_HOST=mysql-service
```

---

50. How Pods Consume ConfigMaps

Two common approaches are:

```
Environment Variables
```

and:

```
Mounted Files
```

---

51. ConfigMap as Environment Variables

Flow:

```
ConfigMap
    ↓
Environment Variables
    ↓
Container
```

Inside the application environment:

```
APP_MODE=production
DATABASE_HOST=mysql-service
```

The Spring Boot application simply reads these environment variables.

It does not need to know that Kubernetes supplied them.

---

52. ConfigMap as Files

Some applications expect configuration files.

Examples:

```
application.properties
nginx.conf
logback.xml
```

A ConfigMap can be mounted into the Pod so that its values appear as files.

```
ConfigMap
    ↓
Mounted Volume
    ↓
Container
    ↓
/config/application.properties
```

---

53. Environment Variable vs Mounted File

For small configuration values:

```
APP_ENV=production
LOG_LEVEL=INFO
DATABASE_HOST=mysql
```

environment variables are convenient.

For full configuration files:

```
application.properties
nginx.conf
logback.xml
```

mounting configuration as files may be more natural.

There is no universal rule that one is always better than the other.

---

54. Secret

A **Secret** is intended for sensitive information.

Examples:

```
Passwords
API Tokens
Authentication Credentials
```

```
Private Keys
Certificates
```

Conceptually:

```
                  Pod
                   |
          ┌────────┴────────┐
          ↓                 ↓
      ConfigMap          Secret

DB_HOST=mysql       DB_USERNAME=admin
LOG_LEVEL=INFO      DB_PASSWORD=*****
```

---

55. Important: Secret Does Not Mean Automatic Encryption

Secret values are often represented using Base64.

Example:

```
password
   ↓
Base64
   ↓
cGFzc3dvcmQ=
```

But Base64 is:

```
Encoding
```

not:

```
Encryption
```

Anyone who has access to the Base64 value can decode it.

Kubernetes Secrets are stored unencrypted in `etcd` by default unless encryption at rest is configured.

Therefore:

> A Kubernetes Secret is an object designed for sensitive data. It is not automatically a complete secret-management vault.

Production environments may additionally use:

```
Encryption at rest
RBAC
External Secrets
Cloud Secret Managers
Vault
KMS
```

---

56. How Pods Consume Secrets

Secrets can be consumed similarly to ConfigMaps.

As Environment Variables

```
Secret
   ↓
DB_PASSWORD Environment Variable
   ↓
Container
```

As Mounted Files

```
Secret
   ↓
Volume
   ↓
/etc/secrets/password
```

The application simply reads the environment variable or file.

---

57. Putting Everything Together

Consider a Spring Boot application:

```
Spring Boot Shop API
```

Docker image:

```
coderarmy/shop-api:v1
```

We want:

```
3 instances
```

So we create:

```
Deployment
   ↓
ReplicaSet
   ↓
┌──────┼──────┐
↓      ↓      ↓
Pod A Pod B Pod C
```

---

58. Add Configuration

All Pods require normal configuration.

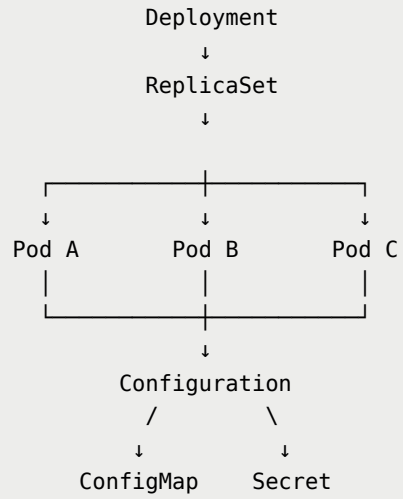
ConfigMap:

```
APP_MODE=production
DB_HOST=mysql-service
LOG_LEVEL=INFO
```

Sensitive configuration goes into a Secret:

```
DB_USER=admin
DB_PASSWORD=*****
```

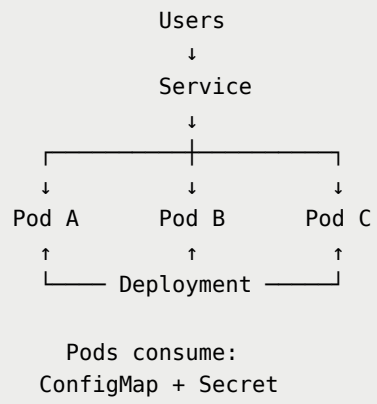
Architecture:



59. Add Networking

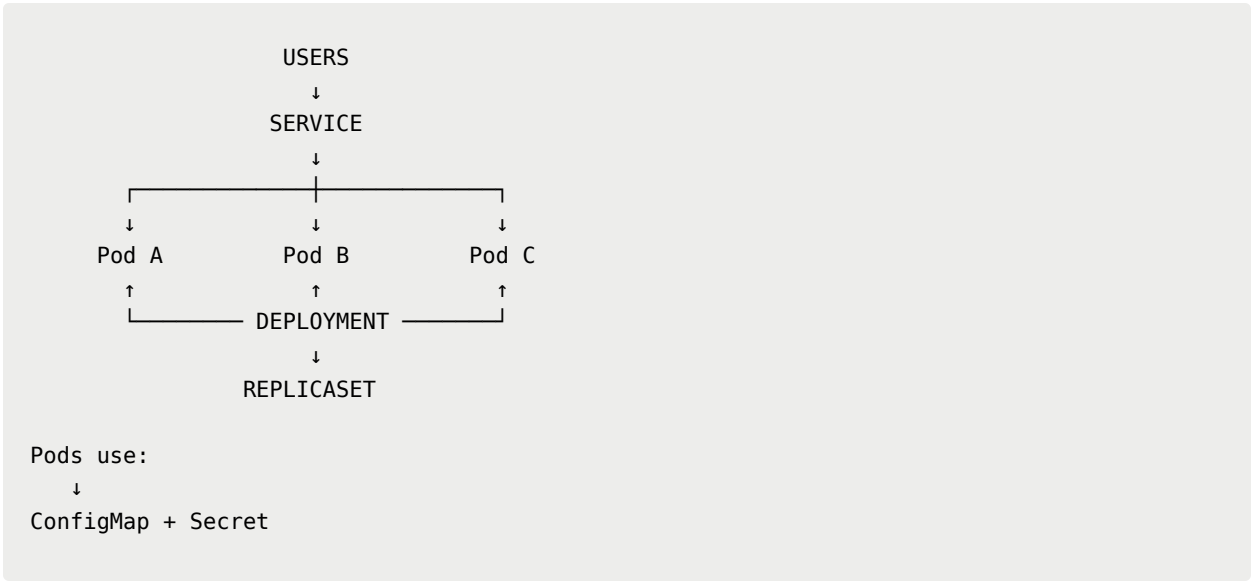
Users also need a stable way to access the application.

Add a Service:



This gives us the basic Kubernetes application architecture.

60. Complete Kubernetes Application Mental Model



Each object has a different responsibility:

Object	Responsibility
Pod	Runs one or more containers
ReplicaSet	Maintains the desired number of matching Pods
Deployment	Manages stateless applications, versions, updates and rollbacks
DaemonSet	Runs a Pod on every relevant Node
StatefulSet	Manages Pods with persistent identity/storage requirements
Service	Provides stable network access to matching Pods
ConfigMap	Stores non-sensitive configuration
Secret	Stores sensitive configuration

61. Kubernetes YAML

A Docker image answers:

```
What should run?
```

Kubernetes YAML answers:

```
How should that image
be run and managed
inside the cluster?
```

For one application, we might create:

```
k8s/

deployment.yaml
service.yaml
configmap.yaml
secret.yaml
```

Keeping resources in separate YAML files is often easier while learning.

Another option is combining multiple Kubernetes resources into one file:

```
app.yaml
```

Both approaches are possible.

Final Mental Model

A useful way to remember the complete flow is:

```
Docker Image
     ↓
Deployment
     ↓
ReplicaSet
     ↓
Pods
```

```
     ↓
Containers
```

Configuration comes from:

```
ConfigMap
+
Secret
```

Network access comes through:

```
Service
```

For specialized workloads:

```
Normal stateless application
        ↓
Deployment
```

```
One Pod per Node
        ↓
DaemonSet
```

```
Persistent identity / storage
        ↓
StatefulSet
```

The core Kubernetes idea throughout all of this remains the same:

```
Define the desired state
        ↓
Kubernetes continuously works
to make the actual state match it
```